从转发树识别突发事件

有人直接转发原帖，也有人沿着已有转发继续传播。把这些关系连成一棵树，就能用模型区分正常事件和突发事件。

20,648	82.25%	0.794
传播图	测试准确率	测试宏平均 F1

看传播形态

直接转发多不多？传播接力了多少层？

给整棵树分类

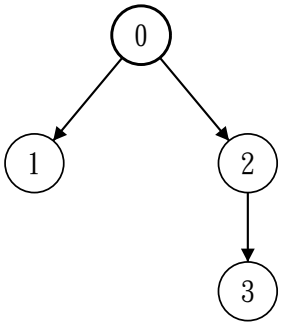
把传播形态和帖子向量交给模型，得到一个突发分数。

用新样本检验

用模型没见过的传播树，检查分类是否准确。

把转发记录读成一棵树

一个节点是一条帖子。一条有向边从来源帖子指向转发帖。



0 是原帖。1 和 2 来自 0，3 来自 2。

edge_index	两行分别是 [0, 0, 2] 和 [1, 2, 3]。
x	每条帖子对应 32 个数值。
label	整张图的类别，0 正常，1 突发。

三个数据集

划分	图数	正常	突发
训练集	14,453	9,633	4,820
验证集	3,097	2,064	1,033
测试集	3,098	2,065	1,033

训练集中的平均传播形态

图特征	正常	突发
节点数	42.44	42.70
最深转发层数	4.90	4.10
最宽一层的节点占比	39.9%	51.1%
直接转发原帖的比例	38.2%	51.6%
没有下游转发的节点占比	63.0%	69.5%

两类图的平均大小接近。突发类更常直接转发原帖，最宽的一层也占了更多节点。每棵树都有 N-1 条边，因此有向图密度等于 1/N，只反映图的大小。

给整棵树判一个类别

逻辑回归先把一张图写成一行数。七项结构特征描述传播形态，32 维节点向量取均值描述整张图的数值特征。两部分可以分开用，也可以拼成 39 维。

图同构网络让相邻节点交换两轮信息，再对节点表示取均值，给整张图打分。用验证集选训练轮次，最后用测试集评估。

测试集　3,098 张图

方法	准确率	宏平均 F1	突发召回率	ROC-AUC
全部判为正常	0.667	0.400	0.000	0.500
仅用节点数	0.667	0.400	0.000	0.488
图结构	0.731	0.664	0.428	0.766
节点向量均值	0.777	0.733	0.556	0.835
图结构与向量均值	0.822	0.794	0.674	0.886
图同构网络	0.796	0.760	0.612	0.850

准确率看全部样本。宏平均 F1 先分别计算两类的 F1，再取平均，因此两类各占一半权重。全部判为正常时，模型漏掉了所有突发样本。

表现最好的方法错在哪里

真实类别	预测正常	预测突发
正常	1,852	213
突发	337	696

图结构与向量均值合用时，识别出 1,033 个突发样本中的 696 个，突发召回率为 67.4%。

自己把实验跑一遍

在仓库根目录开始，使用 Python 3.10 或以上版本。

```
python -m pip install -r requirements.txt
cd experiments/weibo_cascades
python generate_dataset.py --out data
python run_experiments.py
```

如果已有三份 .pt 文件，可以跳过生成步骤，并在最后一条命令后加上 --data-dir 和对应的目录。

运行后看这些文件

文件	内容
metrics.json	事件分类指标和训练记录
event_test_predictions.csv	每张测试图的类别和模型分数
gin_model.pt	验证集选出的模型权重

怎样解读这些结果

这些分数衡量模型在模拟实验中对完整传播树的分类能力。实际传播速度需要时间戳。若要做突发预警，需要确定观测截止时间，只给模型看此前出现的传播记录。